

Most Complex Python and Database Code

The complexity was primarily in the design and decision-making behind the code, rather than simply the number of lines.

1. Most Complex Python Code

Stacking Classifier implementation using Logistic Regression, Decision Tree, and K-Nearest Neighbors as base models, with Logistic Regression as the meta-model.

Although the final implementation is short, choosing the appropriate base models and meta-model was the most challenging part. I selected models with different learning behaviors so that they could provide complementary predictions rather than simply making the same errors.

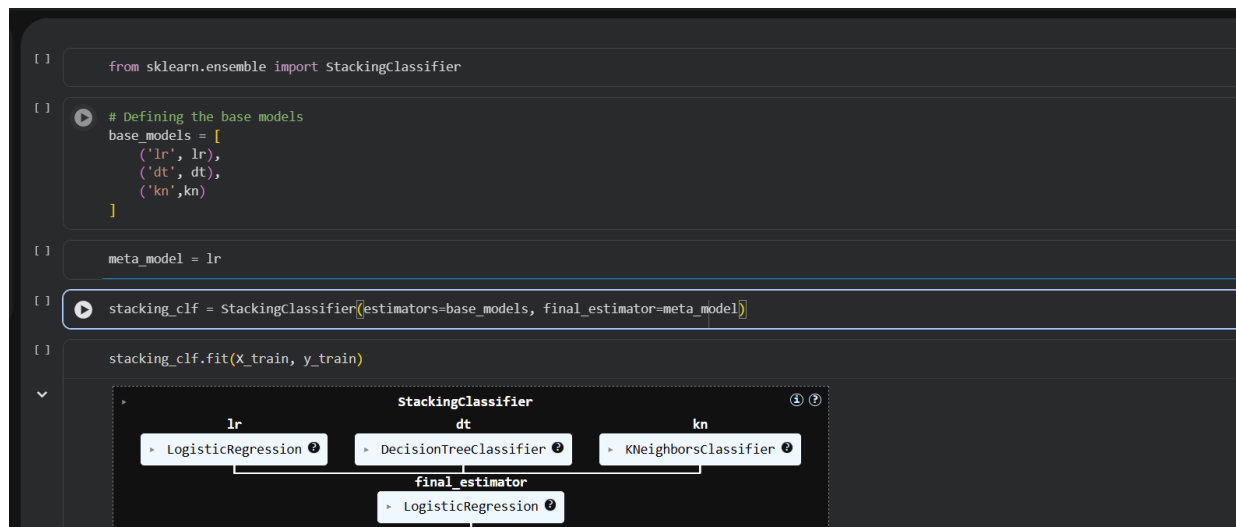


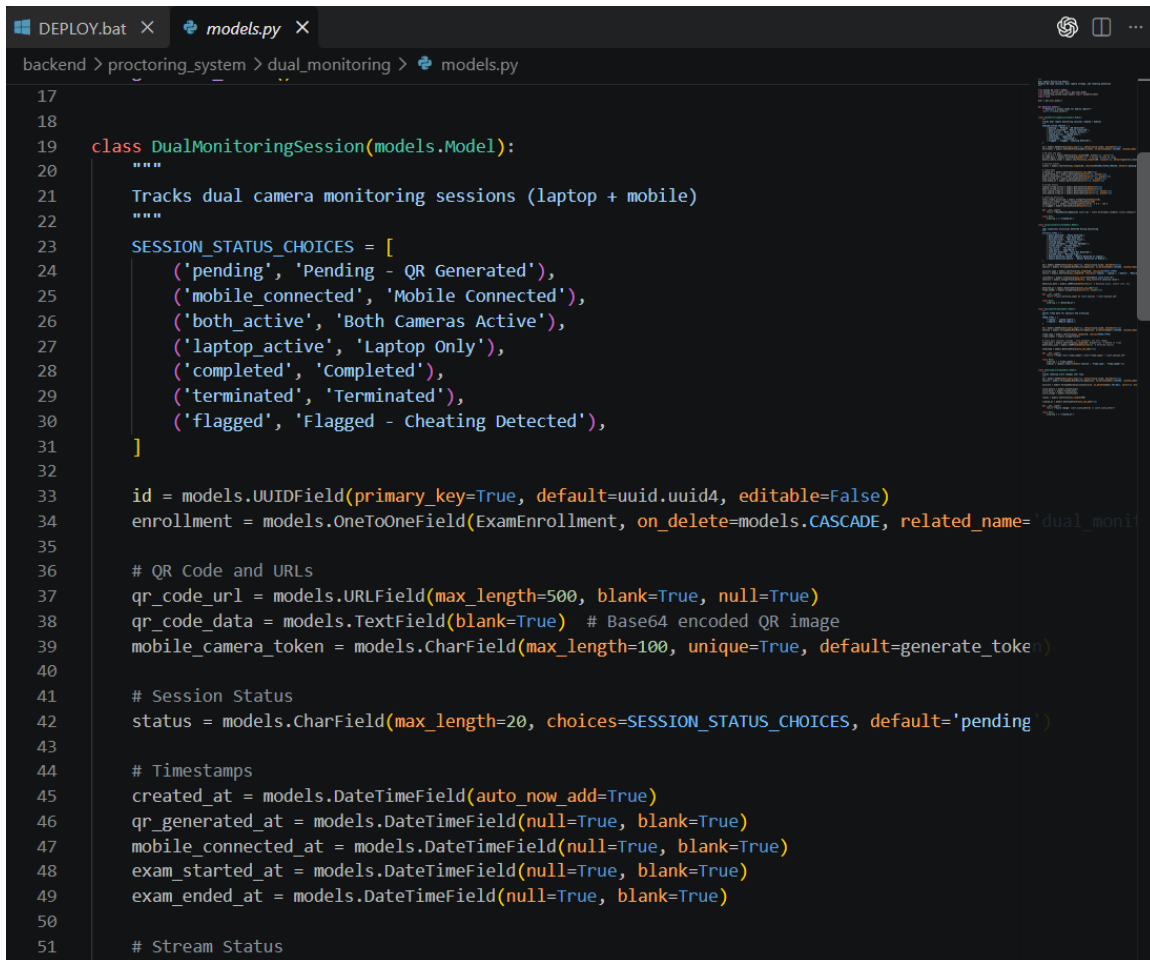
Figure 1: Stacking Classifier implementation

The architecture is: Input Data → Logistic Regression + Decision Tree + KNN → base-model predictions → Logistic Regression meta-model → final prediction.

2. Most Complex Database Code

Django database models for the dual-camera monitoring and suspicious-activity tracking module of my VISION (Visual Intelligence System for Invigilation and Online Monitoring) project.

The challenging part was designing the relationships between an exam enrollment, a monitoring session, and multiple suspicious activities, while also storing AI-detection confidence, severity, and variable detection metadata.

A screenshot of a code editor with a dark theme. The top bar shows two tabs: 'DEPLOY.bat' and 'models.py'. The breadcrumb navigation at the top reads 'backend > proctoring_system > dual_monitoring > models.py'. The code is a Python class definition for 'DualMonitoringSession' which inherits from 'models.Model'. It includes a docstring, a list of session status choices, and various database fields like 'id', 'enrollment', 'qr_code_url', 'qr_code_data', 'mobile_camera_token', 'status', and several timestamps. The code is line-numbered from 17 to 51.

```
17
18
19 class DualMonitoringSession(models.Model):
20     """
21     Tracks dual camera monitoring sessions (laptop + mobile)
22     """
23     SESSION_STATUS_CHOICES = [
24         ('pending', 'Pending - QR Generated'),
25         ('mobile_connected', 'Mobile Connected'),
26         ('both_active', 'Both Cameras Active'),
27         ('laptop_active', 'Laptop Only'),
28         ('completed', 'Completed'),
29         ('terminated', 'Terminated'),
30         ('flagged', 'Flagged - Cheating Detected'),
31     ]
32
33     id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
34     enrollment = models.OneToOneField(ExamEnrollment, on_delete=models.CASCADE, related_name='dual_moni
35
36     # QR Code and URLs
37     qr_code_url = models.URLField(max_length=500, blank=True, null=True)
38     qr_code_data = models.TextField(blank=True) # Base64 encoded QR image
39     mobile_camera_token = models.CharField(max_length=100, unique=True, default=generate_token)
40
41     # Session Status
42     status = models.CharField(max_length=20, choices=SESSION_STATUS_CHOICES, default='pending')
43
44     # Timestamps
45     created_at = models.DateTimeField(auto_now_add=True)
46     qr_generated_at = models.DateTimeField(null=True, blank=True)
47     mobile_connected_at = models.DateTimeField(null=True, blank=True)
48     exam_started_at = models.DateTimeField(null=True, blank=True)
49     exam_ended_at = models.DateTimeField(null=True, blank=True)
50
51     # Stream Status
```

Figure 2: The hard part for the database

The database relationship is:

Exam Enrollment → One Dual Monitoring Session → Many Suspicious Activities.

The UUID provides unique session identification, the OneToOneField connects the exam enrollment to its monitoring session, and the ForeignKey allows a session to contain multiple suspicious events. The JSONField provides flexibility for storing variable AI-detection metadata.